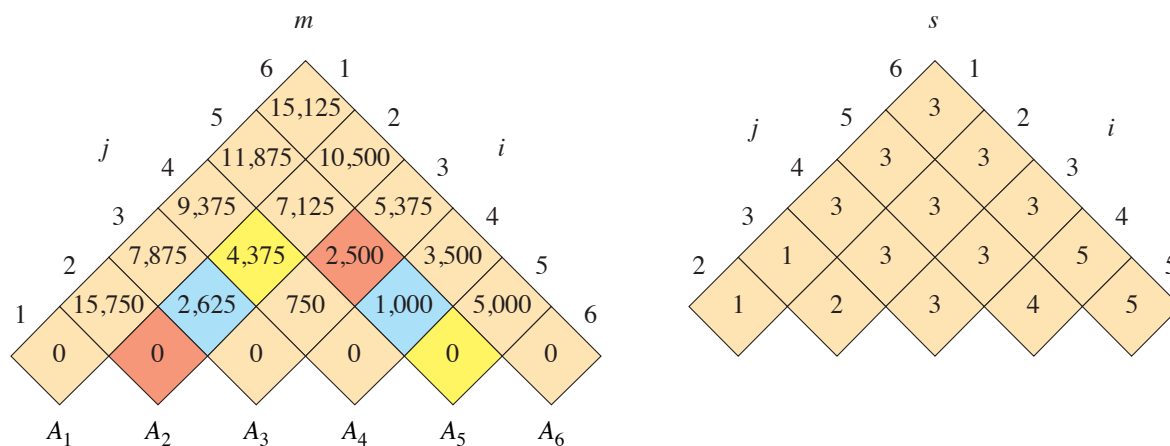


cost  $m[i, j]$ . Equation (14.7) tells us that to compute the cost of matrix product  $A_{i:j}$ , first the costs of the products  $A_{i:k}$  and  $A_{k+1:j}$  need to have been computed for all  $k = i, i + 1, \dots, j - 1$ . The chain  $A_i A_{i+1} \dots A_j$  consists of  $j - i + 1$  matrices, and the chains  $A_i A_{i+1} \dots A_k$  and  $A_{k+1} A_{k+2} \dots A_j$  consist of  $k - i + 1$  and  $j - k$  matrices, respectively. Since  $k < j$ , a chain of  $k - i + 1$  matrices consists of fewer than  $j - i + 1$  matrices. Likewise, since  $k \geq i$ , a chain of  $j - k$  matrices consists of fewer than  $j - i + 1$  matrices. Thus, the algorithm should fill in the table  $m$  from shorter matrix chains to longer matrix chains. That is, for the subproblem of optimally parenthesizing the chain  $A_i A_{i+1} \dots A_j$ , it makes sense to consider the subproblem size as the length  $j - i + 1$  of the chain.

Now, let's see how the MATRIX-CHAIN-ORDER procedure fills in the  $m[i, j]$  entries in order of increasing chain length. Lines 2–3 initialize  $m[i, i] = 0$  for  $i = 1, 2, \dots, n$ , since any matrix chain with just one matrix requires no scalar multiplications. In the **for** loop of lines 4–12, the loop variable  $l$  denotes the length of matrix chains whose minimum costs are being computed. Each iteration of this loop uses recurrence (14.7) to compute  $m[i, i + l - 1]$  for  $i = 1, 2, \dots, n - l + 1$ . In the first iteration,  $l = 2$ , and so the loop computes  $m[i, i + 1]$  for  $i = 1, 2, \dots, n - 1$ : the minimum costs for chains of length  $l = 2$ . The second time through the loop, it computes  $m[i, i + 2]$  for  $i = 1, 2, \dots, n - 2$ : the minimum costs for chains of length  $l = 3$ . And so on, ending with a single matrix chain of length  $l = n$  and computing  $m[1, n]$ . When lines 7–12 compute an  $m[i, j]$  cost, this cost depends only on table entries  $m[i, k]$  and  $m[k + 1, j]$ , which have already been computed.

Figure 14.5 illustrates the  $m$  and  $s$  tables, as filled in by the MATRIX-CHAIN-ORDER procedure on a chain of  $n = 6$  matrices. Since  $m[i, j]$  is defined only for  $i \leq j$ , only the portion of the table  $m$  on or above the main diagonal is used. The figure shows the table rotated to make the main diagonal run horizontally. The matrix chain is listed along the bottom. Using this layout, the minimum cost  $m[i, j]$  for multiplying a subchain  $A_i A_{i+1} \dots A_j$  of matrices appears at the intersection of lines running northeast from  $A_i$  and northwest from  $A_j$ . Reading across, each diagonal in the table contains the entries for matrix chains of the same length. MATRIX-CHAIN-ORDER computes the rows from bottom to top and from left to right within each row. It computes each entry  $m[i, j]$  using the products  $p_{i-1} p_k p_j$  for  $k = i, i + 1, \dots, j - 1$  and all entries southwest and southeast from  $m[i, j]$ .

A simple inspection of the nested loop structure of MATRIX-CHAIN-ORDER yields a running time of  $O(n^3)$  for the algorithm. The loops are nested three deep, and each loop index ( $l$ ,  $i$ , and  $k$ ) takes on at most  $n - 1$  values. Exercise 14.2-5 asks you to show that the running time of this algorithm is in fact also  $\Omega(n^3)$ . The algorithm requires  $\Theta(n^2)$  space to store the  $m$  and  $s$  tables. Thus, MATRIX-CHAIN-ORDER is much more efficient than the exponential-time method of enumerating all possible parenthesizations and checking each one.



**Figure 14.5** The  $m$  and  $s$  tables computed by MATRIX-CHAIN-ORDER for  $n = 6$  and the following matrix dimensions:

| matrix    | $A_1$          | $A_2$          | $A_3$         | $A_4$         | $A_5$          | $A_6$          |
|-----------|----------------|----------------|---------------|---------------|----------------|----------------|
| dimension | $30 \times 35$ | $35 \times 15$ | $15 \times 5$ | $5 \times 10$ | $10 \times 20$ | $20 \times 25$ |

The tables are rotated so that the main diagonal runs horizontally. The  $m$  table uses only the main diagonal and upper triangle, and the  $s$  table uses only the upper triangle. The minimum number of scalar multiplications to multiply the 6 matrices is  $m[1, 6] = 15,125$ . Of the entries that are not tan, the pairs that have the same color are taken together in line 9 when computing

$$\begin{aligned}
 m[2, 5] &= \min \begin{cases} m[2, 2] + m[3, 5] + p_1 p_2 p_5 = 0 + 2500 + 35 \cdot 15 \cdot 20 = 13,000, \\ m[2, 3] + m[4, 5] + p_1 p_3 p_5 = 2625 + 1000 + 35 \cdot 5 \cdot 20 = 7125, \\ m[2, 4] + m[5, 5] + p_1 p_4 p_5 = 4375 + 0 + 35 \cdot 10 \cdot 20 = 11,375 \end{cases} \\
 &= 7125.
 \end{aligned}$$

#### Step 4: Constructing an optimal solution

Although MATRIX-CHAIN-ORDER determines the optimal number of scalar multiplications needed to compute a matrix-chain product, it does not directly show how to multiply the matrices. The table  $s[1:n-1, 2:n]$  provides the information needed to do so. Each entry  $s[i, j]$  records a value of  $k$  such that an optimal parenthesization of  $A_i A_{i+1} \cdots A_j$  splits the product between  $A_k$  and  $A_{k+1}$ . The final matrix multiplication in computing  $A_{1:n}$  optimally is  $A_{1:s[1,n]} A_{s[1,n]+1:n}$ . The  $s$  table contains the information needed to determine the earlier matrix multiplications as well, using recursion:  $s[1, s[1, n]]$  determines the last matrix multiplication when computing  $A_{1:s[1,n]}$  and  $s[s[1, n] + 1, n]$  determines the last matrix multiplication when computing  $A_{s[1,n]+1:n}$ . The recursive procedure PRINT-OPTIMAL-PARENS on the facing page prints an optimal parenthesization of the matrix chain product  $A_i A_{i+1} \cdots A_j$ , given the  $s$  table computed by MATRIX-CHAIN-ORDER and the in-

trices  $i$  and  $j$ . The initial call  $\text{PRINT-OPTIMAL-PARENS}(s, 1, n)$  prints an optimal parenthesization of the full matrix chain product  $A_1 A_2 \cdots A_n$ . In the example of Figure 14.5, the call  $\text{PRINT-OPTIMAL-PARENS}(s, 1, 6)$  prints the optimal parenthesization  $((A_1(A_2 A_3))((A_4 A_5) A_6))$ .

```

PRINT-OPTIMAL-PARENS( $s, i, j$ )
1  if  $i == j$ 
2      print " $A$ " $i$ 
3  else print "("
4      PRINT-OPTIMAL-PARENS( $s, i, s[i, j]$ )
5      PRINT-OPTIMAL-PARENS( $s, s[i, j] + 1, j$ )
6      print ")"

```

## Exercises

### 14.2-1

Find an optimal parenthesization of a matrix-chain product whose sequence of dimensions is  $\langle 5, 10, 3, 12, 5, 50, 6 \rangle$ .

### 14.2-2

Give a recursive algorithm  $\text{MATRIX-CHAIN-MULTIPLY}(A, s, i, j)$  that actually performs the optimal matrix-chain multiplication, given the sequence of matrices  $\langle A_1, A_2, \dots, A_n \rangle$ , the  $s$  table computed by  $\text{MATRIX-CHAIN-ORDER}$ , and the indices  $i$  and  $j$ . (The initial call is  $\text{MATRIX-CHAIN-MULTIPLY}(A, s, 1, n)$ .) Assume that the call  $\text{RECTANGULAR-MATRIX-MULTIPLY}(A, B)$  returns the product of matrices  $A$  and  $B$ .

### 14.2-3

Use the substitution method to show that the solution to the recurrence (14.6) is  $\Omega(2^n)$ .

### 14.2-4

Describe the subproblem graph for matrix-chain multiplication with an input chain of length  $n$ . How many vertices does it have? How many edges does it have, and which edges are they?

### 14.2-5

Let  $R(i, j)$  be the number of times that table entry  $m[i, j]$  is referenced while computing other table entries in a call of  $\text{MATRIX-CHAIN-ORDER}$ . Show that the total number of references for the entire table is

$$\sum_{i=1}^n \sum_{j=i}^n R(i, j) = \frac{n^3 - n}{3}.$$

(Hint: You may find equation (A.4) on page 1141 useful.)

### 14.2-6

Show that a full parenthesization of an  $n$ -element expression has exactly  $n - 1$  pairs of parentheses.

---

## 14.3 Elements of dynamic programming

Although you have just seen two complete examples of the dynamic-programming method, you might still be wondering just when the method applies. From an engineering perspective, when should you look for a dynamic-programming solution to a problem? In this section, we'll examine the two key ingredients that an optimization problem must have in order for dynamic programming to apply: optimal substructure and overlapping subproblems. We'll also revisit and discuss more fully how memoization might help you take advantage of the overlapping-subproblems property in a top-down recursive approach.

### Optimal substructure

The first step in solving an optimization problem by dynamic programming is to characterize the structure of an optimal solution. Recall that a problem exhibits *optimal substructure* if an optimal solution to the problem contains within it optimal solutions to subproblems. When a problem exhibits optimal substructure, that gives you a good clue that dynamic programming might apply. (As Chapter 15 discusses, it also might mean that a greedy strategy applies, however.) Dynamic programming builds an optimal solution to the problem from optimal solutions to subproblems. Consequently, you must take care to ensure that the range of subproblems you consider includes those used in an optimal solution.

Optimal substructure was key to solving both of the previous problems in this chapter. In Section 14.1, we observed that the optimal way of cutting up a rod of length  $n$  (if Serling Enterprises makes any cuts at all) involves optimally cutting up the two pieces resulting from the first cut. In Section 14.2, we noted that an optimal parenthesization of the matrix chain product  $A_i A_{i+1} \cdots A_j$  that splits the product between  $A_k$  and  $A_{k+1}$  contains within it optimal solutions to the problems of parenthesizing  $A_i A_{i+1} \cdots A_k$  and  $A_{k+1} A_{k+2} \cdots A_j$ .

You will find yourself following a common pattern in discovering optimal substructure:

1. You show that a solution to the problem consists of making a choice, such as choosing an initial cut in a rod or choosing an index at which to split the matrix chain. Making this choice leaves one or more subproblems to be solved.
2. You suppose that for a given problem, you are given the choice that leads to an optimal solution. You do not concern yourself yet with how to determine this choice. You just assume that it has been given to you.
3. Given this choice, you determine which subproblems ensue and how to best characterize the resulting space of subproblems.
4. You show that the solutions to the subproblems used within an optimal solution to the problem must themselves be optimal by using a “cut-and-paste” technique. You do so by supposing that each of the subproblem solutions is not optimal and then deriving a contradiction. In particular, by “cutting out” the nonoptimal solution to each subproblem and “pasting in” the optimal one, you show that you can get a better solution to the original problem, thus contradicting your supposition that you already had an optimal solution. If an optimal solution gives rise to more than one subproblem, they are typically so similar that you can modify the cut-and-paste argument for one to apply to the others with little effort.

To characterize the space of subproblems, a good rule of thumb says to try to keep the space as simple as possible and then expand it as necessary. For example, the space of subproblems for the rod-cutting problem contained the problems of optimally cutting up a rod of length  $i$  for each size  $i$ . This subproblem space worked well, and it was not necessary to try a more general space of subproblems.

Conversely, suppose that you tried to constrain the subproblem space for matrix-chain multiplication to matrix products of the form  $A_1 A_2 \cdots A_j$ . As before, an optimal parenthesization must split this product between  $A_k$  and  $A_{k+1}$  for some  $1 \leq k < j$ . Unless you can guarantee that  $k$  always equals  $j - 1$ , you will find that you have subproblems of the form  $A_1 A_2 \cdots A_k$  and  $A_{k+1} A_{k+2} \cdots A_j$ . Moreover, the latter subproblem does not have the form  $A_1 A_2 \cdots A_j$ . To solve this problem by dynamic programming, you need to allow the subproblems to vary at “both ends.” That is, both  $i$  and  $j$  need to vary in the subproblem of parenthesizing the product  $A_i A_{i+1} \cdots A_j$ .

Optimal substructure varies across problem domains in two ways:

1. how many subproblems an optimal solution to the original problem uses, and
2. how many choices you have in determining which subproblem(s) to use in an optimal solution.

In the rod-cutting problem, an optimal solution for cutting up a rod of size  $n$  uses just one subproblem (of size  $n - i$ ), but we have to consider  $n$  choices for  $i$  in order to determine which one yields an optimal solution. Matrix-chain multiplication for the subchain  $A_i A_{i+1} \cdots A_j$  serves an example with two subproblems and  $j - i$  choices. For a given matrix  $A_k$  where the product splits, two subproblems arise—parenthesizing  $A_i A_{i+1} \cdots A_k$  and parenthesizing  $A_{k+1} A_{k+2} \cdots A_j$ —and we have to solve *both* of them optimally. Once we determine the optimal solutions to subproblems, we choose from among  $j - i$  candidates for the index  $k$ .

Informally, the running time of a dynamic-programming algorithm depends on the product of two factors: the number of subproblems overall and how many choices you look at for each subproblem. In rod cutting, we had  $\Theta(n)$  subproblems overall, and at most  $n$  choices to examine for each, yielding an  $O(n^2)$  running time. Matrix-chain multiplication had  $\Theta(n^2)$  subproblems overall, and each had at most  $n - 1$  choices, giving an  $O(n^3)$  running time (actually, a  $\Theta(n^3)$  running time, by Exercise 14.2-5).

Usually, the subproblem graph gives an alternative way to perform the same analysis. Each vertex corresponds to a subproblem, and the choices for a subproblem are the edges incident from that subproblem. Recall that in rod cutting, the subproblem graph has  $n$  vertices and at most  $n$  edges per vertex, yielding an  $O(n^2)$  running time. For matrix-chain multiplication, if you were to draw the subproblem graph, it would have  $\Theta(n^2)$  vertices and each vertex would have degree at most  $n - 1$ , giving a total of  $O(n^3)$  vertices and edges.

Dynamic programming often uses optimal substructure in a bottom-up fashion. That is, you first find optimal solutions to subproblems and, having solved the subproblems, you find an optimal solution to the problem. Finding an optimal solution to the problem entails making a choice among subproblems as to which you will use in solving the problem. The cost of the problem solution is usually the subproblem costs plus a cost that is directly attributable to the choice itself. In rod cutting, for example, first we solved the subproblems of determining optimal ways to cut up rods of length  $i$  for  $i = 0, 1, \dots, n - 1$ , and then we determined which of these subproblems yielded an optimal solution for a rod of length  $n$ , using equation (14.2). The cost attributable to the choice itself is the term  $p_i$  in equation (14.2). In matrix-chain multiplication, we determined optimal parenthesizations of subchains of  $A_i A_{i+1} \cdots A_j$ , and then we chose the matrix  $A_k$  at which to split the product. The cost attributable to the choice itself is the term  $p_{i-1} p_k p_j$ .

Chapter 15 explores “greedy algorithms,” which have many similarities to dynamic programming. In particular, problems to which greedy algorithms apply have optimal substructure. One major difference between greedy algorithms and dynamic programming is that instead of first finding optimal solutions to subproblems and then making an informed choice, greedy algorithms first make a “greedy” choice—the choice that looks best at the time—and then solve a resulting subprob-

lem, without bothering to solve all possible related smaller subproblems. Surprisingly, in some cases this strategy works!

### Subtleties

You should be careful not to assume that optimal substructure applies when it does not. Consider the following two problems whose input consists of a directed graph  $G = (V, E)$  and vertices  $u, v \in V$ .

**Unweighted shortest path:**<sup>5</sup> Find a path from  $u$  to  $v$  consisting of the fewest edges. Such a path must be simple, since removing a cycle from a path produces a path with fewer edges.

**Unweighted longest simple path:** Find a simple path from  $u$  to  $v$  consisting of the most edges. (Without the requirement that the path must be simple, the problem is undefined, since repeatedly traversing a cycle creates paths with an arbitrarily large number of edges.)

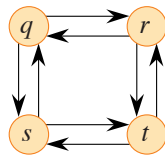
The unweighted shortest-path problem exhibits optimal substructure. Here's how. Suppose that  $u \neq v$ , so that the problem is nontrivial. Then, any path  $p$  from  $u$  to  $v$  must contain an intermediate vertex, say  $w$ . (Note that  $w$  may be  $u$  or  $v$ .) Then, we can decompose the path  $u \xrightarrow{p} v$  into subpaths  $u \xrightarrow{p_1} w \xrightarrow{p_2} v$ . The number of edges in  $p$  equals the number of edges in  $p_1$  plus the number of edges in  $p_2$ . We claim that if  $p$  is an optimal (i.e., shortest) path from  $u$  to  $v$ , then  $p_1$  must be a shortest path from  $u$  to  $w$ . Why? As suggested earlier, use a “cut-and-paste” argument: if there were another path, say  $p'_1$ , from  $u$  to  $w$  with fewer edges than  $p_1$ , then we could cut out  $p_1$  and paste in  $p'_1$  to produce a path  $u \xrightarrow{p'_1} w \xrightarrow{p_2} v$  with fewer edges than  $p$ , thus contradicting  $p$ 's optimality. Likewise,  $p_2$  must be a shortest path from  $w$  to  $v$ . Thus, to find a shortest path from  $u$  to  $v$ , consider all intermediate vertices  $w$ , find a shortest path from  $u$  to  $w$  and a shortest path from  $w$  to  $v$ , and choose an intermediate vertex  $w$  that yields the overall shortest path. Section 23.2 uses a variant of this observation of optimal substructure to find a shortest path between every pair of vertices on a weighted, directed graph.

You might be tempted to assume that the problem of finding an unweighted longest simple path exhibits optimal substructure as well. After all, if we decompose a longest simple path  $u \xrightarrow{p} v$  into subpaths  $u \xrightarrow{p_1} w \xrightarrow{p_2} v$ , then mustn't  $p_1$  be a longest simple path from  $u$  to  $w$ , and mustn't  $p_2$  be a longest simple path from  $w$  to  $v$ ? The answer is no! Figure 14.6 supplies an example. Consider the

---

<sup>5</sup> We use the term “unweighted” to distinguish this problem from that of finding shortest paths with weighted edges, which we shall see in Chapters 22 and 23. You can use the breadth-first search technique of Chapter 20 to solve the unweighted problem.





**Figure 14.6** A directed graph showing that the problem of finding a longest simple path in an unweighted directed graph does not have optimal substructure. The path  $q \rightarrow r \rightarrow t$  is a longest simple path from  $q$  to  $t$ , but the subpath  $q \rightarrow r$  is not a longest simple path from  $q$  to  $r$ , nor is the subpath  $r \rightarrow t$  a longest simple path from  $r$  to  $t$ .

path  $q \rightarrow r \rightarrow t$ , which is a longest simple path from  $q$  to  $t$ . Is  $q \rightarrow r$  a longest simple path from  $q$  to  $r$ ? No, for the path  $q \rightarrow s \rightarrow t \rightarrow r$  is a simple path that is longer. Is  $r \rightarrow t$  a longest simple path from  $r$  to  $t$ ? No again, for the path  $r \rightarrow q \rightarrow s \rightarrow t$  is a simple path that is longer.

This example shows that for longest simple paths, not only does the problem lack optimal substructure, but you cannot necessarily assemble a “legal” solution to the problem from solutions to subproblems. If you combine the longest simple paths  $q \rightarrow s \rightarrow t \rightarrow r$  and  $r \rightarrow q \rightarrow s \rightarrow t$ , you get the path  $q \rightarrow s \rightarrow t \rightarrow r \rightarrow q \rightarrow s \rightarrow t$ , which is not simple. Indeed, the problem of finding an unweighted longest simple path does not appear to have any sort of optimal substructure. No efficient dynamic-programming algorithm for this problem has ever been found. In fact, this problem is NP-complete, which—as we shall see in Chapter 34—means that we are unlikely to find a way to solve it in polynomial time.

Why is the substructure of a longest simple path so different from that of a shortest path? Although a solution to a problem for both longest and shortest paths uses two subproblems, the subproblems in finding the longest simple path are not *independent*, whereas for shortest paths they are. What do we mean by subproblems being independent? We mean that the solution to one subproblem does not affect the solution to another subproblem of the same problem. For the example of Figure 14.6, we have the problem of finding a longest simple path from  $q$  to  $t$  with two subproblems: finding longest simple paths from  $q$  to  $r$  and from  $r$  to  $t$ . For the first of these subproblems, we chose the path  $q \rightarrow s \rightarrow t \rightarrow r$ , which used the vertices  $s$  and  $t$ . These vertices cannot appear in a solution to the second subproblem, since the combination of the two solutions to subproblems yields a path that is not simple. If vertex  $t$  cannot be in the solution to the second problem, then there is no way to solve it, since  $t$  is required to be on the path that forms the solution, and it is not the vertex where the subproblem solutions are “spliced” together (that vertex being  $r$ ). Because vertices  $s$  and  $t$  appear in one subproblem solution, they cannot appear in the other subproblem solution. One of them must be in the solution to the other subproblem, however, and an optimal solution requires both.



Thus, we say that these subproblems are not independent. Looked at another way, using resources in solving one subproblem (those resources being vertices) renders them unavailable for the other subproblem.

Why, then, are the subproblems independent for finding a shortest path? The answer is that by nature, the subproblems do not share resources. We claim that if a vertex  $w$  is on a shortest path  $p$  from  $u$  to  $v$ , then we can splice together *any* shortest path  $u \xrightarrow{p_1} w$  and *any* shortest path  $w \xrightarrow{p_2} v$  to produce a shortest path from  $u$  to  $v$ . We are assured that, other than  $w$ , no vertex can appear in both paths  $p_1$  and  $p_2$ . Why? Suppose that some vertex  $x \neq w$  appears in both  $p_1$  and  $p_2$ , so that we can decompose  $p_1$  as  $u \xrightarrow{p_{ux}} x \rightsquigarrow w$  and  $p_2$  as  $w \rightsquigarrow x \xrightarrow{p_{xv}} v$ . By the optimal substructure of this problem, path  $p$  has as many edges as  $p_1$  and  $p_2$  together. Let's say that  $p$  has  $e$  edges. Now let us construct a path  $p' = u \xrightarrow{p_{ux}} x \xrightarrow{p_{xv}} v$  from  $u$  to  $v$ . Because we have excised the paths from  $x$  to  $w$  and from  $w$  to  $x$ , each of which contains at least one edge, path  $p'$  contains at most  $e - 2$  edges, which contradicts the assumption that  $p$  is a shortest path. Thus, we are assured that the subproblems for the shortest-path problem are independent.

The two problems examined in Sections 14.1 and 14.2 have independent subproblems. In matrix-chain multiplication, the subproblems are multiplying subchains  $A_i A_{i+1} \cdots A_k$  and  $A_{k+1} A_{k+2} \cdots A_j$ . These subchains are disjoint, so that no matrix could possibly be included in both of them. In rod cutting, to determine the best way to cut up a rod of length  $n$ , we looked at the best ways of cutting up rods of length  $i$  for  $i = 0, 1, \dots, n-1$ . Because an optimal solution to the length- $n$  problem includes just one of these subproblem solutions (after cutting off the first piece), independence of subproblems is not an issue.

### Overlapping subproblems

The second ingredient that an optimization problem must have for dynamic programming to apply is that the space of subproblems must be “small” in the sense that a recursive algorithm for the problem solves the same subproblems over and over, rather than always generating new subproblems. Typically, the total number of distinct subproblems is a polynomial in the input size. When a recursive algorithm revisits the same problem repeatedly, we say that the optimization problem has *overlapping subproblems*.<sup>6</sup> In contrast, a problem for which a divide-and-

---

<sup>6</sup> It may seem strange that dynamic programming relies on subproblems being both independent and overlapping. Although these requirements may sound contradictory, they describe two different notions, rather than two points on the same axis. Two subproblems of the same problem are independent if they do not share resources. Two subproblems are overlapping if they are really the same subproblem that occurs as a subproblem of different problems.

